Abstract

The artificial intelligence industry has spent recent years making foundational models increasingly capable. Modern models can reason, write code, call tools, retrieve information, operate browsers, interact with APIs, and execute complex multi-step tasks. In response, developers have built autonomous agents around these models, adding layers of planning, memory, tool use, retries, reflection, routing, and orchestration.

Recent advances in artificial intelligence have produced increasingly capable models and sophisticated agent harnesses. However, these systems continue to struggle with reliability because they conflate cognitive generation with state management, placing the burden of planning, execution, and recovery directly onto the agent.

Despite these sophisticated AI harnesses, autonomous systems continue to suffer from fundamental reliability bottlenecks. The root cause is not a lack of better prompts, more robust tools, or advanced orchestration; rather, it is an architectural flaw: the agent is asked to manage cognition, state, execution, and recovery simultaneously. Managing these divergent responsibilities is an infrastructural role that belongs to a dedicated operating system, not the agent itself.

This paper introduces CognitiveOS, an architectural layer and operational runtime that solves this fundamental problem through explicit state ownership, validation, and deterministic state transition semantics. By positioning models and agents as proposers rather than authorities, CognitiveOS establishes a shared governance substrate. This closed cognitive runtime manages the complete lifecycle of cognition—planning, execution, observation, evaluation, learning, persistence, and recovery—ensuring systemic coherence as a system interacts with the dynamic external world.

1. Introduction

Human understanding is not constructed solely by observing the environment. Much of what humans know is learned through active intervention. Consider a person who encounters a door and attempts to open it:

- **Initial State:** The person believes a door is locked and tries the handle; the door fails to open.
- **First Revision:** The person attempts to use a key that ultimately fails, shifting their understanding to recognize that the door may be locked and the key may be incorrect.
- **Resolution:** A subsequent attempt with an alternative key opens the door, yielding an understanding far richer than the initial observation.

The resulting knowledge is not merely a collection of observations, but accumulated experience regarding the relationship between action and consequence. Humans continuously learn not only what exists in the world, but what happens when they act upon it.

When an individual encounters an unfamiliar machine, they begin with a conceptual model (e.g., that pressing a specific button starts the device). Pressing the button successfully reinforces this belief. However, encountering an unexpected response in a different context—such as discovering the machine will not start when a safety cover is open . The human constructs this nuanced understanding through a sequence of interactions or actions

Humans rarely act without expectation in biology. This is called a stimulus a human or any organism will take action in response to stimulus with a particular goal in mind driven by the stimulus in systems language this can be an event that is used to trigger a response but the idea is the same the world around us is not static and is continuously changing we respond to changes in the world and the response in turn might change the world itself. However When an action yields a discrepancy between expected outcomes and reality (e.g., a door remaining closed despite an attempt to open it), that discrepancy drives additional cognition. Rather than merely recording a failure, the actor asks why—considering whether the door is locked, blocked, broken, or misdirectionally manipulated. This requirement justifies an explicit relationship between prediction, execution, observation, and comparison within any cognitive runtime. In our architecture we call entities that respond to changes in the world model represented by state or may cause change in the world state by taking action or responding to changes in the world state as Actors . An actor can be an enterprise or person represented in the world who when takes an action may or may not make change to the world model as a result of interacting with the world but will update its local belief system based on its interactions The Action thus functions simultaneously as an attempt to achieve a goal, an intervention on the world, an experiment about the world, and a source of evidence for future cognition.

Similarly while a language model can answer static queries about hypothetical actions, a persistent actor must evaluate discrepancies, recognize incomplete understanding, and modify its future behavior. Without this recursive transition, an agent merely executes instructions against a static or repeatedly reconstructed model without genuine adaptation.

This is where realtime grounding of the context becomes important . In current AI Architectures Grounding is frequently treated as supplying an AI system with static environmental context.

For a true actor, grounding must be continuous across the operational loop: Because every interaction changes the state from which the next interaction begins, the loop has no fixed beginning or end. If an action does not meaningfully affect an agent's future cognitive state, actor-level learning does not occur. A persistent actor must carry forward its intent, beliefs, observations, predictions, execution logs, successes, failures, and newly formed expectations as interconnected runtime concerns rather than isolated services.

In multi-agent environments (such as a manufacturing facility where one actor initiates maintenance on a machine another actor believes is available), local representations quickly become stale. A governed cognitive architecture requires distributed mechanisms to propagate validated state changes so that multiple actors maintain coherent operational alignment within a shared world.

Why This Matters for Cognitive Architectures

The ultimate purpose of a Cognitive Operating System (CognitiveOS) is not simply to provision agents with memory, tools, planning interfaces, or execution harnesses. It is to provide the governed lifecycle through which an actor interacts with reality, reconciles expectations, updates its beliefs, and closes the operational loop:

The Fundamental Principle

Intelligent behavior requires that an agent cannot remain an external observer querying a static world model. It must participate as an actor whose interactions continuously transform the cognitive baseline from which future decisions emerge.

> **Core Principle:** An intelligent actor is not defined merely by its ability to act on the world. It is defined by its ability to change as a consequence of acting on the world.

The compelling intuition that reliable operation in a changing environment requires more than a language model's statistical representation of reality has led to multiple directions of research including world models . However, a world model taken independently does not solve the broader architectural problem of building a reliable cognitive system. While a world model answers what the system believes about the world, a Cognitive Operating System must govern a much larger set of runtime questions, encompassing task intent, permissions, validation, actual outcomes, evaluation, belief updates, and future actions. A world model represents the environment; a cognitive runtime governs what the system does with that representation.

Accurate world representation remains immensely valuable for embodied and long-running AI systems. However, the architectural takeaway is structural: a world model must participate *within* cognition rather than become synonymous with it. The correct relationship integrates the world model into a persistent, governed loop spanning state creation, planning, validation, execution, observation, comparison, learning, and belief updates.

The Architectural Mistake

The primary architectural pitfall in modern AI engineering is assuming that solving one isolated dimension of cognition solves the entire system:

- Better world models solve environmental awareness.
- Better memory systems solve historical retrieval.
- Better planners solve action generation.
- Better agent harnesses solve component coordination.

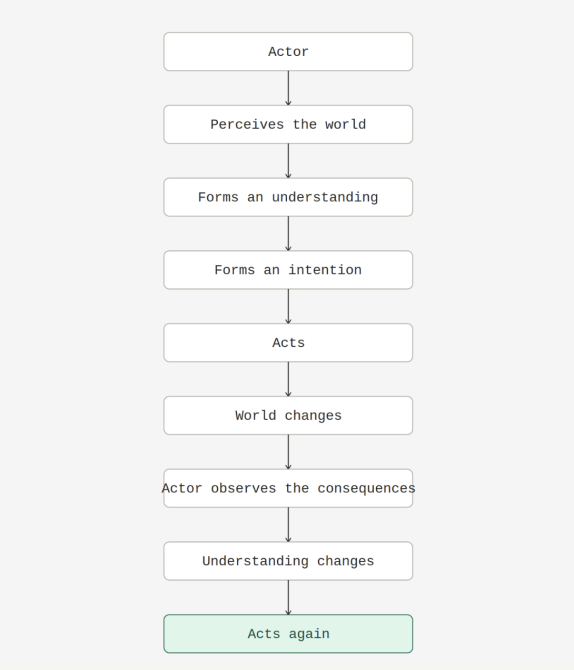
However, a Cognitive Operating System must solve how the entire cognitive process evolves coherently over time under a unified authority, connecting world states, task states, beliefs, plans, and execution evidence into a unified operational loop.

To address this challenge, this paper introduces the Cognitive Operating System (CognitiveOS). Rather than replacing models or agents, CognitiveOS provides a shared governance substrate within which they operate as proposers. The operating system retains ownership of an authoritative state, governing what is allowed to become part of that state or reach the external world.

2. Actors, Interaction, and the Evolution of Understanding

What Is an Actor?

An actor is an entity that participates in a world through a continuous cycle of observation, intention, action, and consequence. Humans, robots, autonomous software agents, and broader business processes can all function as actors. What distinguishes an actor from a passive information system is not simply the ability to produce an output; an actor can change the world and subsequently be changed by its interaction with that world. An actor therefore exists in a dynamic relationship with its environment:



This loop is fundamental. An actor does not merely possess a static model of the world; its interaction with the world continuously changes the model it uses to decide what to do next.

Actor Coordination and Multi-Agent Topology

Actor coordination is the mechanism through which multiple autonomous actors collaborate to achieve a shared objective. Each actor maintains its local belief state and reasons independently. Coordination is achieved through the exchange of natural-language messages rather than direct state sharing, with the **Planetary Runtime** providing the underlying communication infrastructure while each actor retains absolute responsibility for its own reasoning and decision-making.

The coordination architecture is governed by the following core principles:

- Every actor is strictly autonomous.
- Every actor owns its local belief state exclusively.
- Actors communicate exclusively using natural-language messages.
- No actor may directly modify another actor's belief state.
- The global world state is updated solely through Planetary Cycles.
- Negotiations are dynamic and LLM-driven rather than hardcoded.
- Coordination sessions are fully observable through negotiation traces.
- Coordination mechanisms adopt game-theoretic frameworks aimed at achieving Nash equilibrium.

Coordination Participants and Responsibilities

A coordination request typically involves an originating actor, one or more affiliate actors, the Planetary Runtime, **SittingFace**, and the Global World State.

Originating Actor

The originating actor owns the coordination session. Its responsibilities include:

- Understanding the primary objective.
- Determining which societies are relevant.
- Resolving affiliations and initiating negotiations.
- Aggregating responses, deciding whether additional coordination is required, and determining when the objective has been successfully achieved.

Constraint: The originating actor does not modify the beliefs of other actors.

Affiliate Actors

Affiliate actors reason independently. Each affiliate:

- Receives a natural-language request via the runtime.
- Executes its internal cognitive loop.
- Updates only its own local belief state.
- Generates an execution trace and returns the result.

Constraint: Affiliate actors never directly modify another actor's state.

Actor Location

Every actor exists at a specific location within the world map. Location is represented as a hierarchical spatial containment graph. Every location belongs to a parent location, allowing the runtime to reason about locality, containment, reachability, and context at multiple levels of abstraction.The hierarchy is extensible. Additional spatial layers may be inserted without changing actor cognition, provided they preserve the parent-child containment relationship.

The hierarchy is:



The location hierarchy enables the runtime to answer questions such as:

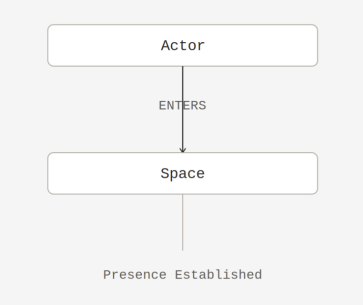
- Where is the actor?
- Which actors are nearby?
- Which resources are within the current building?
- Which events occurred within this city?
- Which policies apply at this location?
- Which actors are within communication range?

- Which societies operate within this location?

The runtime can traverse upward or downward through the hierarchy to determine the appropriate spatial scope for reasoning and coordination.

Presence

Presence represents an actor's current occupancy of a Space. An actor establishes Presence by entering a Space. Likewise, Presence is removed when the actor exits the Space. Presence is therefore dynamic and continuously changes as actors move throughout the world.



Presence is independent of the actor's permanent affiliations or memberships. Every Space must have at least one associated Society. During actor discovery, the Planetary Runtime first determines the actor's current Presence.

It then:

1. identifies the current Space;
2. retrieves the Societies associated with that Space;
3. establishes the actor's temporary memberships;
4. discovers eligible affiliates within those societies;
5. applies policy and trust filtering; and
6. begins coordination.

Within the CognitiveOS architecture, the boundary between environmental context and active participation is intentionally fluid: a physical location can simultaneously function as a spatial container (a Space) and an active participant (an Actor).

Selecting appropriate autonomous agents for complex user requests extends beyond traditional capability-matching algorithms. In multi-agent ecosystems, a technically capable agent is not necessarily the optimal actor given user-specific constraints, historical interactions, and environmental contexts. By integrating **presence** (dynamic actor participation), evolving **user preferences**, and **interaction history**, CognitiveOS transitions agent discovery from static capability mapping to relational, state-aware agent resolution. This architecture treats agent selection as a meta-cognitive process, establishing persistent, relationship-driven interactions between users and cognitive actors

Preferences in CognitiveOS are modeled as dynamic components of the cognitive state rather than static configurations. Users develop persistent preferences through explicit configuration and implicit feedback loops over repeated interactions

for a physical location—such as a smart warehouse zone, a secure cleanroom, a server facility, or a manufacturing floor—operates within the runtime, it exhibits two distinct operational roles:

As a Space: It provides the physical boundaries, environmental context, and jurisdictional domain that hosts other actors, automatically provisioning temporary society memberships to any entity establishing Presence within its perimeter.

As an Actor: It maintains its own local belief state, executes an independent cognitive loop, monitors environmental sensors, enforces local invariants, and participates directly in multi-actor coordination sessions.

Presence within the CognitiveOS architecture extends beyond spatial coordinates to define an actor's immediate coordination neighborhood. Rather than recording only a static location, the runtime maintains a comprehensive, indexed history of presence across operational spaces. Presence therefore defines the actor's immediate coordination neighborhood. Rather than storing only the current location, the runtime maintains the complete history of presence.

This history records:

- visited spaces;
- entry time;
- exit time;
- duration;
- associated societies;
- nearby actors;
- activities performed;
- goals being pursued;
- negotiation sessions;
- important observations.

Presence therefore becomes a behavioral history rather than a simple location history. Presence history is indexed by SittingFace. Before an LLM invocation, the runtime may retrieve patterns

such as:

- commonly visited spaces;
- recurring movement patterns;
- historical collaborators;
- preferred coordination styles;
- frequently used resources;
- recurring activities.

These learned patterns help ground reasoning without requiring explicit rules. Treating physical locations as first-class actors embedded within Spaces enables powerful architectural capabilities:

- **Self-Reporting Infrastructure:** Facilities and physical zones can directly negotiate resource allocation, report maintenance states, or raise safety vetoes during multi-agent planning.
- **Environmental Agency:** A location-actor can observe perturbations within its boundaries, update its local beliefs regarding equipment availability or safety thresholds, and publish world perturbation events to the Planetary Runtime.
- **Unified Governance:** Because locations participate as actors governed by local societies, physical constraints (such as maximum occupancy, hazardous material containment, or temperature limits) are enforced through standard coordination and negotiation mechanics rather than hardcoded edge checks.

Creating agent selection as a contextual resolution problem elevates routing from an infrastructure concern to a core cognitive decision (meta-cognition). Before executing a task, the CognitiveOS runtime evaluates intent, required capabilities, experiential history, trust matrices, and real-time presence states. The resulting architecture constructs a persistent cognitive society rather than an interchangeable pool of stateless tools

Society

A Society represents the community responsible for governing, coordinating, or operating within a given physical or logical Space. Examples include Building Societies, Warehouse Societies, Manufacturing Societies, City Societies, Campus Societies, and Emergency Response Societies. A single Space may contain multiple overlapping societies. For instance, a physical

Building space might simultaneously encompass:

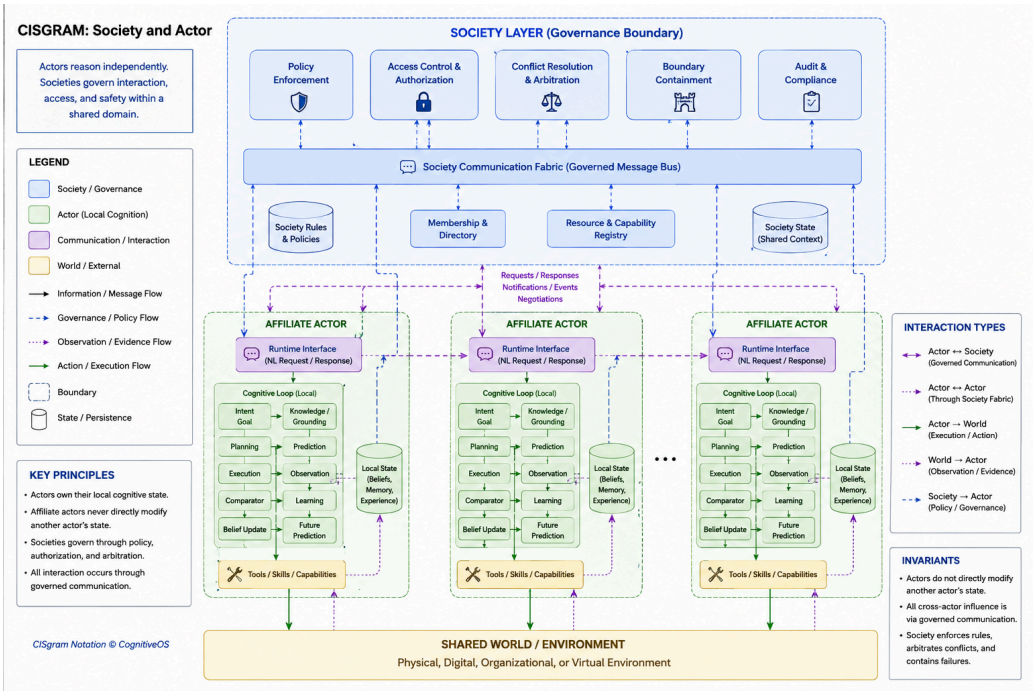
- Security Society
- Maintenance Society
- Manufacturing Society
- Safety Society

These societies collectively define the potential coordination network available to any actor operating within that Space.Beyond acting as collaborative communities for coordination, a

Society functions as an authoritative **governance boundary**. Within its designated operational domain, a society enforces structural constraints, regulatory policies, access controls, and behavioral guardrails on all participating actors.

The society occupies a critical architectural boundary within CognitiveOS, acting as the bridge between internal agent reasoning and external world interaction. This creates a distinct three-level operational hierarchy:

This separation allows CognitiveOS to scale to highly complex organizations of autonomous actors without requiring them to collapse into a single, monolithic cognitive process.



The Three-Level Inquiry Boundary

- Inside the Actor (Cognition):
    - What do I believe?
    - What am I trying to accomplish?
    - What should I do?
- At the Society Boundary (Governance):
    - Am I allowed to do it?
    - Does this action conflict with another actor?
    - Does it satisfy policy?
    - What resources may I access?
    - What happens if this action affects another actor?
    - How should conflicts and failures be contained?
- Beyond the Boundary (Reality):
    - What actually happens in the shared world?

The structural evolution of multi-actor systems within CognitiveOS does not follow a simple linear progression from single to multi-agent frameworks. Instead, it scales through structured containment: This architecture ensures that adding actors to a system does not mean surrendering operational control.

- Each actor maintains absolute independence, reasoning from its own experience and local beliefs while learning continuously from interactions with the world.
- When those actors participate within a common society, their proposals and interactions become subject to a shared governance boundary.

This design enables autonomous actors to organize into sophisticated teams, organizations, fleets, and societies without devolving into either a rigid centralized intelligence or an uncontrolled swarm of independent agents.

> **Summary Principle:** Actors provide cognition. Societies provide governance. The world provides consequences. A Cognitive Operating System supplies the runtime infrastructure required to keep all three seamlessly connected.

Key Governance Functions

- **Policy Enforcement:** Societies mandate specific operational rules, safety protocols, and compliance checks that actors must satisfy before executing actions or committing state changes.
- **Access Control and Authorization:** The society boundary determines which actors possess the legal or operational clearance to interact with specific resources, tools, or other members within that domain.
- **Conflict Resolution and Arbitration:** When multi-agent negotiations result in deadlocks or competing objectives, the governing society provides the structural protocols and consensus rules required to arbitrate disputes.
- **Boundary Containment:** By defining clear jurisdictional limits, societies ensure that cascading failures, unauthorized state perturbations, or policy violations do not propagate unchecked across the broader system.

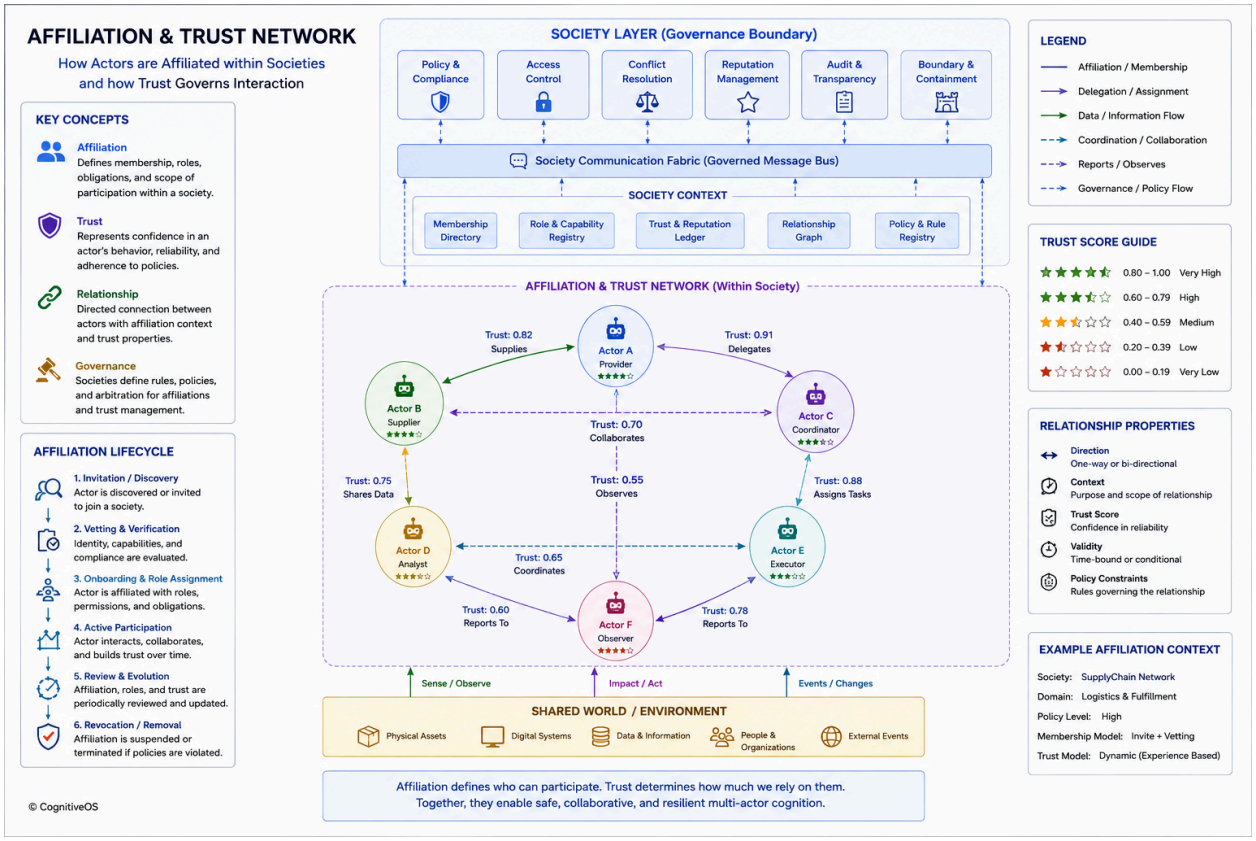
Society memberships serve as the operational bridge between spatial location and coordination capability. These societies define the potential coordination network available within any given Space.

When an actor establishes **Presence** within a Space, the runtime automatically provisionally enrolls that actor as a temporary member of every Society associated with that Space. This temporary membership exists solely for the duration of the actor's active Presence.

Conversely, when the actor departs the Space, its Presence record is updated, and all associated temporary memberships are immediately revoked This mechanism ensures that actor capabilities and governance boundaries adapt dynamically to physical or logical relocation

without requiring manual registration, explicit configuration updates, or centralized administrative intervention.

Affiliation and Trust Network



An affiliation establishes the context in which an actor participates in a larger system. An actor may belong to a team, participate in a society, operate within an organization, or be associated with a particular operational domain. These relationships establish the actor's position within the social structure of the world.

For example, an actor may be a member of an engineering team, a robot may be affiliated with an assembly society, or an autonomous service may operate as part of a procurement organization. These relationships provide information that can become relevant during grounding and planning. An actor therefore needs to reason not only about the state of the environment, but also about its position in the society

The Affiliation Graph models structural relationships between actors, societies, enterprises, organizations, and memberships. The relationship between the originating actor and its affiliates

are defined via an affiliation graph The Affiliation Graph models the structural relationships between actors, societies, enterprises, organizations, and memberships.

It answers questions such as:

- Which societies does an actor belong to?
- Which enterprises is an actor affiliated with?
- Which actors belong to the same society?
- Which actors may legally communicate?
- Which organizations can participate in a negotiation?

The graph is relatively stable and changes only when affiliations or memberships change. The Trust Network overlays the Affiliation Graph, representing the directional degree of confidence one actor has in another. Unlike static structural affiliations, trust is dynamic and evolves through continuous operational experience.

Trust provides a mechanism through which this history can influence future cognition.
Consider an actor that repeatedly delegates deliveries to a second actor. If those deliveries consistently succeed, the first actor accumulates evidence supporting confidence in the second actor. If the second actor repeatedly fails, that confidence may decrease.

The resulting relationship can therefore evolve:
Interaction → Outcome → Evidence → Trust Update → Future Decision

Trust is consequently not intended to represent an unconditional assertion that another actor is correct. A trusted actor may still provide incorrect information. Instead, trust contributes to the evaluation of information and actions by providing contextual evidence about their expected reliability.

Trust influences the interpretation of evidence; it does not replace evidence.
An observation from a highly trusted actor may receive greater contextual weight than an observation from an actor with no relevant history, but the receiving actor remains responsible for determining how that information should affect its own beliefs. The Trust Network overlays the Affiliation Graph It represents the degree of confidence that one actor has in another. Unlike affiliations, trust is dynamic and evolves through experience.

Trust values are directional and asymmetric. Factors continuously shaping trust include successful and failed negotiations, execution accuracy, policy compliance, historical reliability, response quality, consistency, timeliness, and demonstrated expertise. Trust is continuously updated as actors interact.

Factors influencing trust include:

- successful negotiations;
- failed negotiations;
- execution accuracy;

- policy compliance;
- historical reliability;
- response quality;
- consistency;
- timeliness;
- Expertise.

Trust should evolve over time through learning rather than static configuration.
These graphs are intentionally distinct.

Affiliation establishes social structure. Trust establishes confidence.

An actor may be affiliated with another actor or society without necessarily trusting all information originating from that entity. Conversely, an actor may develop a high degree of trust in another actor through repeated successful interactions even when the two actors do not share the same organizational affiliation.

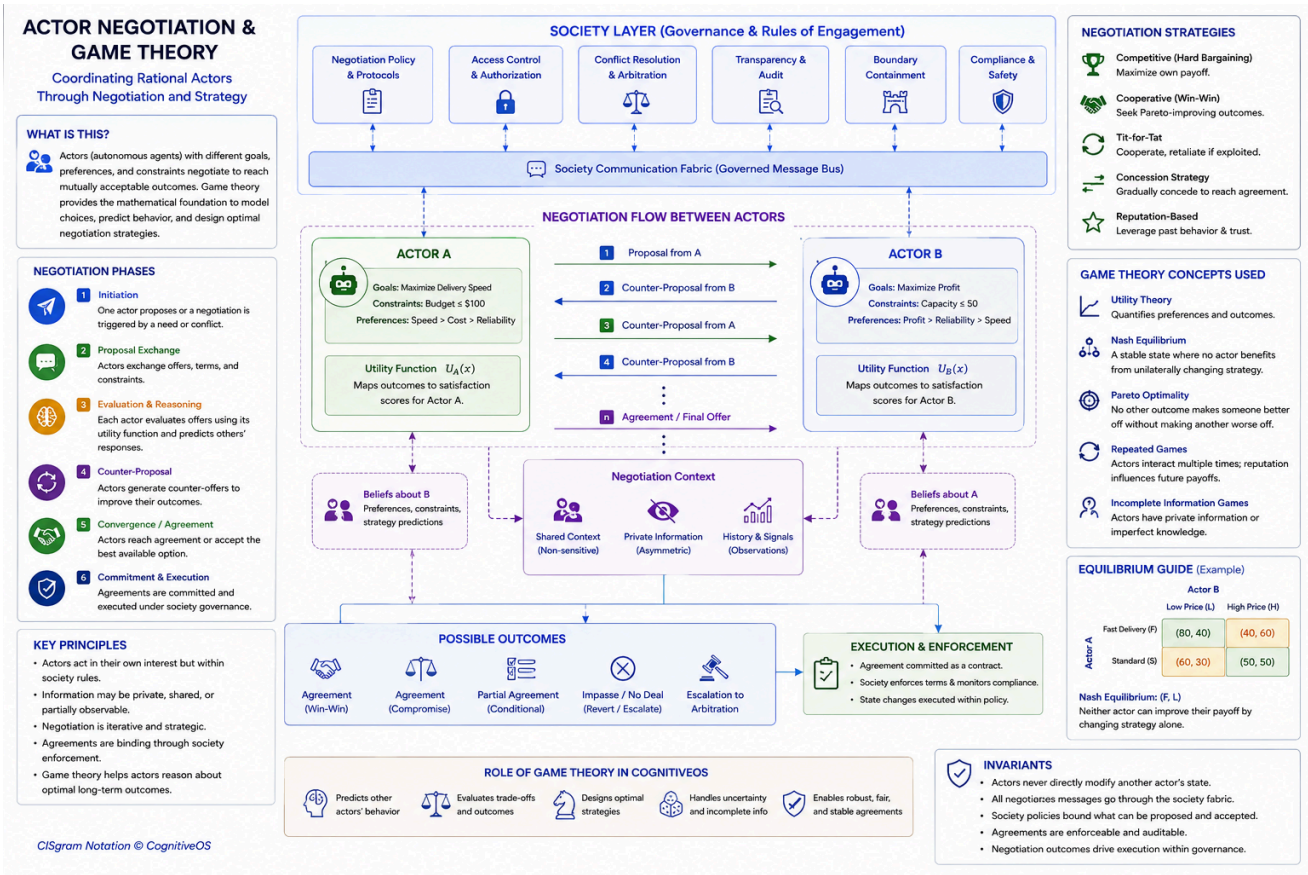
When an actor receives information from another actor, the information cannot always be interpreted independently of its source. The receiving actor may have prior experience with that source, knowledge of its role, knowledge of its capabilities, or evidence concerning the reliability of its previous actions.

The importance of the Trust Graph becomes more apparent when actors operate over long periods. Human trust is not generally established once and then treated as permanently fixed. It evolves through experience. Repeated successful interactions strengthen confidence, while repeated failures can weaken it.

Autonomous actors operating in persistent environments require an analogous capability. Suppose an actor repeatedly interacts with three service providers. All three may currently report that they can perform a task. However, the actor's experience may indicate that one provider consistently succeeds, another succeeds intermittently, and the third frequently fails. A static capability representation might conclude that all three providers are capable.

A socially informed cognitive system must be able to distinguish between **capability** and **experienced reliability**. This distinction allows future planning and delegation to incorporate not only what an actor *can* do, but what the system has learned about how reliably that actor performs the relevant task. The Trust Graph therefore connects social experience to future cognition.

Actor Negotiation and game theory



Autonomous multi-agent systems often struggle to maintain consistency when environmental changes occur asynchronously with internal actor reasoning. We introduce the dual-event architecture of CognitiveOS, which formally separates objective environmental updates from subjective belief evolution.

An actor participating in a CognitiveOS society optimizes for local goals while operating within shared environmental constraints. When resource contention occurs, resolution requires moving beyond single-actor optimization to strategic interaction In contention where proposals conflict and no voluntary agreement is emerging framing the resolved outcome as a Nash equilibrium means: whatever allocation the arbitration mechanism settles on, no actor could have done better for itself by unilaterally proposing something different, given the other actor's strategy and the rules of the arbitration process. That's a *stability* guarantee, not a fairness one . It isn't necessarily either. It's just that nobody has an incentive to keep fighting. However Negotiation Creates Social Experience

Negotiation begins when the Cognitive OS identifies an interaction between actors whose intended actions are potentially conflicting, mutually dependent, or resource constrained. Each actor contributes its current goal, beliefs, available capabilities, constraints, preferences, and the consequences it associates with possible actions. These representations provide the basis for determining whether the actors can cooperate directly, whether one actor must defer to another, or whether an explicit negotiation is required.

The system models these interactions using principles from **game theory**. An actor is not assumed to optimize a global objective. It operates according to its own goals and utility, subject to the information and capabilities available to it. The Cognitive OS therefore evaluates possible joint outcomes rather than simply selecting the locally optimal action for a single actor.

A negotiation can be represented as a game in which:

- **Actors** represent autonomous participants with distinct goals and interests.
- **Strategies** represent the actions or plans available to each actor.
- **Constraints** represent resource, temporal, policy, capability, or environmental limitations.
- **Utilities** represent the value or cost an actor assigns to possible outcomes.
- **Beliefs** represent each actor's current model of the world and of other actors.
- **Information** determines what each actor knows about the state, capabilities, and intentions of the other participants.
- **Outcomes** represent the resulting world state produced by the combination of actor strategies.

The purpose of negotiation is not necessarily to produce a compromise. The objective is to find an executable outcome that satisfies the relevant constraints while maximizing the collective or actor-specific utility permitted by the governing policies.

For example, two actors may require the same machine at overlapping times. Actor A may consider its task urgent, while Actor B may have a higher-value task but a more flexible deadline. Rather than allowing both actors to independently schedule the machine and discover the conflict during execution, the Cognitive OS identifies the shared resource as a point of contention. The actors can then evaluate alternative schedules, exchange relevant information, and determine an allocation that produces the best feasible outcome.

Negotiation can therefore produce several classes of outcomes:

1. **Cooperation** — actors coordinate their plans because a joint strategy produces greater utility than independent execution.
2. **Prioritization** — one actor receives access to a constrained resource because its objective has greater priority under the governing policy.
3. **Trade-off** — actors exchange concessions, such as time, resources, or capabilities, to reach a mutually acceptable outcome.

4. **Delegation** — one actor transfers part of its objective to another actor whose capabilities make execution more efficient.
5. **Deferral** — an actor postpones execution because another actor has a stronger immediate claim on a shared resource.
6. **Escalation** — no acceptable equilibrium can be reached autonomously, requiring a higher-level actor or human authority to resolve the conflict.

This negotiation process is grounded in the Cognitive OS's broader representation of the world. Actors do not negotiate against an abstract or static environment. Existing system state, observations, world changes, policies, historical interactions, and current resource availability constrain the set of strategies that can actually be executed. The negotiation process therefore operates over **grounded possibilities**, not hypothetical actions disconnected from the current world.

A critical property of the system is that negotiation is **stateful**. The outcome of a previous interaction can influence future negotiations through the actor's accumulated history, trust, preferences, and observed behavior. An actor that repeatedly fulfills commitments may receive greater trust in future negotiations, while an actor that consistently violates agreements may require stronger verification or reduced autonomy.

This connects game theory with the Cognitive OS **trust network and affiliation graph**. Trust provides an estimate of whether another actor is likely to fulfill a negotiated commitment, while affiliation represents the structural relationships between actors and organizations. Together, these signals influence which negotiation partners are selected, how much information is shared, and which proposed outcomes are considered credible.

Negotiation is therefore not a separate optimization layer placed above the operating system. It is part of the cognitive loop:

Perceive → Ground → Model Actors → Generate Strategies → Negotiate → Select Outcome → Execute → Observe → Update Beliefs and Trust

The result is an operating environment in which autonomous actors can coordinate without requiring every interaction to be explicitly programmed in advance. The system provides the mechanisms for actors to reason about one another, evaluate competing objectives, negotiate over shared constraints, and adapt their future behavior based on the outcomes of those interactions.

In this model, **game theory becomes an execution primitive for multi-actor cognition**. It provides the formal language for reasoning about conflict, cooperation, incentives, strategic behavior, and equilibrium, while the Cognitive OS supplies the world state, beliefs, relationships, capabilities, policies, and execution mechanisms required to make those decisions actionable.

Actors apply game-theoretic abstractions to evaluate strategic environments. Rather than asking solely which action achieves my goal

An actor can learn:

- which actors negotiate reliably;
- which actors honor commitments;
- which proposals tend to succeed;
- which compromises produce good outcomes;
- which strategies lead to deadlock;
- which resources consistently generate conflict.

This experience can affect future trust and strategy.

Because actors interact with each other repeatedly rather than once, the negotiation subsystem supports a strategy layer distinct from any single negotiation instance: Competitive (hard bargaining, maximize own payoff), Cooperative (seek Pareto-improving outcomes), Tit-for-Tat (cooperate by default, mirror the other actor's last move — retaliate if exploited, reward if reciprocated), Concession (gradual movement toward agreement), and Reputation-Based (weight proposals by the counterparty's trust history).

This is where the system's repeated-game character becomes architecturally relevant: an actor's choice of strategy is not fixed per negotiation but can itself update based on the outcome history captured by the Negotiation Context's "history and signals" channel and by the Trust Graph. An actor that learns a counterparty reliably escalates to arbitration rather than settling has learned something about that counterparty's bargaining posture

System Invariants

Five invariants hold regardless of negotiation outcome or strategy in use:

- actors never directly modify another actor's cognitive state;
- all negotiation messages pass through the Society Communication Fabric, never peer-to-peer;
- society policy bounds what can be legally proposed or accepted in the first place;
- agreements, once committed, are enforceable and auditable;
- and negotiation outcomes drive execution strictly within governance, never outside it.

These are what distinguish this subsystem from a conventional multi-agent messaging layer — the presence of enforceable consequence and persistent state, not merely the exchange of proposals.

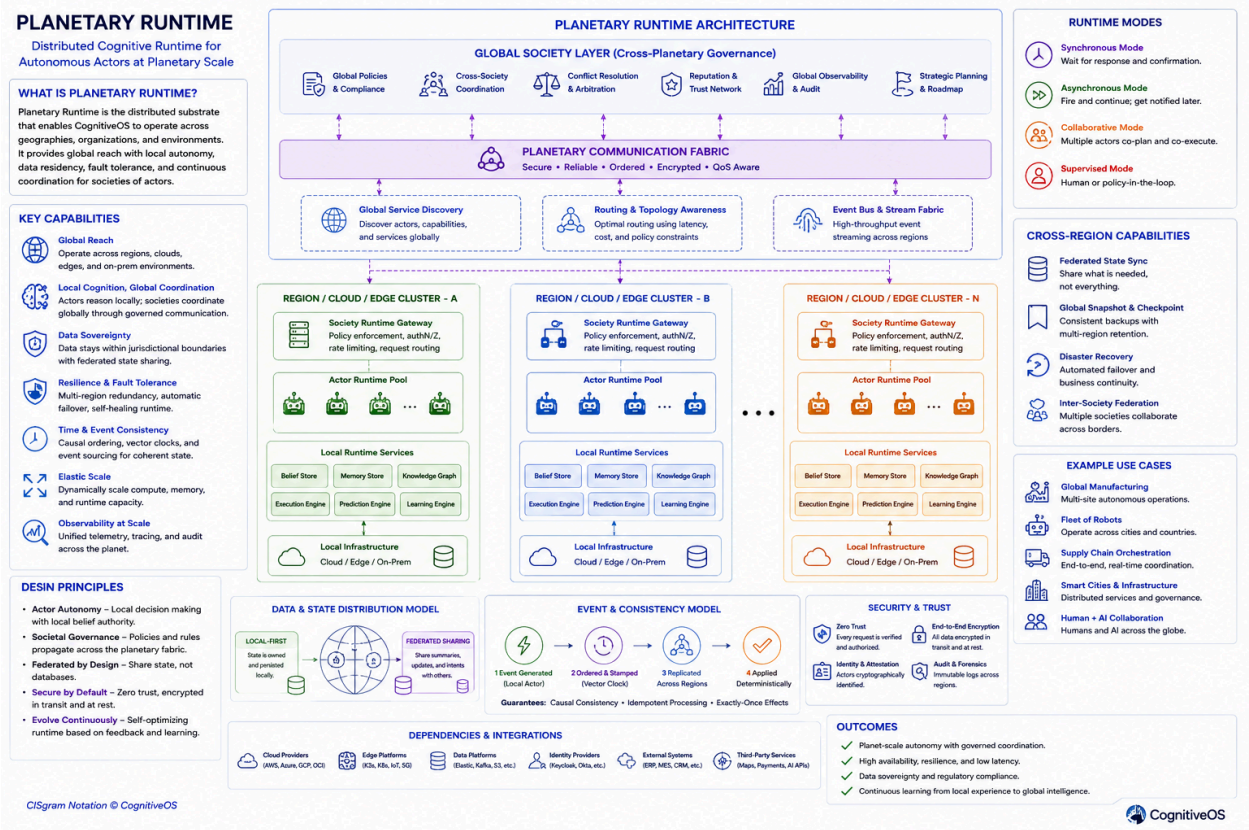
Actor Negotiation Protocol

Every negotiation session must follow a deterministic discovery and authorization sequence:

Objective → Society → Affiliation → Policy → Trust → Ranking → Prompt → Negotiation

- **Identify relevant societies**: Determine which societies, organizations, communities, or institutional domains are relevant to the objective.
- **Traverse the Affiliation Graph:** Starting from those societies, traverse affiliation relationships to discover actors that are structurally connected and potentially eligible to participate.
- **Apply policy and authorization filters:** Remove actors that are not authorized, capable, compliant, or permitted to participate in the objective.
- **Consult the Trust Network:** Evaluate the remaining actors using historical trust, reliability, prior interactions, commitments, and other trust signals.
- **Rank eligible participants :** Produce an ordered participant set using relevance + affiliation + trust + capability + current context.
- **Generate grounded natural-language prompts:** Construct the negotiation request using the actual objective, current world state, actor context, applicable policies, and known relationship information.
- **Begin negotiation:** Initiate the actor-to-actor negotiation using the selected participants and grounded negotiation context.

Planetary Runtime



A critical failure mode in distributed AI architectures is the emergence of the monolithic "super-agent"—a centralized orchestrator that absorbs all planning, routing, and decision-making logic, reducing individual agents to stateless function executors. Such designs undermine autonomy, complicate horizontal scaling, and concentrate systemic vulnerability.

CognitiveOS prevents this architectural collapse by establishing a rigorous demarcation line:The division of labor between the runtime infrastructure and the autonomous actor is governed by distinct functional domains

The **Planetary Runtime** is the distributed coordination layer of the Cognitive OS. Its responsibility is **not to reason, negotiate, or make business decisions**. Instead, it provides the infrastructure required for autonomous actors to discover one another, communicate, coordinate, and synchronize their activities across the system.

As multi-agent systems scale from isolated clusters to distributed ecosystems, centralized orchestration models inevitably fail due to computational bottlenecks and single-point decision failures. We formalize the **Separation of Coordination and Cognition** in **CognitiveOS**, an architectural boundary that bifurcates systemic management from localized reasoning. The **Planetary Runtime** is strictly restricted to structural orchestration, participant discovery,

message routing, and state synchronization, whereas individual **Actors** retain sole ownership over internal beliefs, utility optimization, game-theoretic negotiation, and decision-making. This separation ensures that collective behavior emerges from decentralized negotiation rather than centralized command-and-control.

The **Planetary Runtime** is foundational. It is society-agnostic and provides the base computing fabric, event queues, perturbation loops, and transactional state engines. It answers the fundamental operational questions:

- How are messages routed reliably across a distributed network?
- When do planetary ticks occur, and how is state synchronized via the Global World Knowledge Graph?
- How are asynchronous events and *Déjà Vu* re-evaluation triggers managed?

The Planetary Runtime is analogous to a **distributed operating-system kernel**. It manages the mechanics of coordination while preserving the autonomy of each actor as an independent cognitive entity. The runtime is responsible for the **transport and coordination mechanics** of the actor ecosystem.

Responsibilities

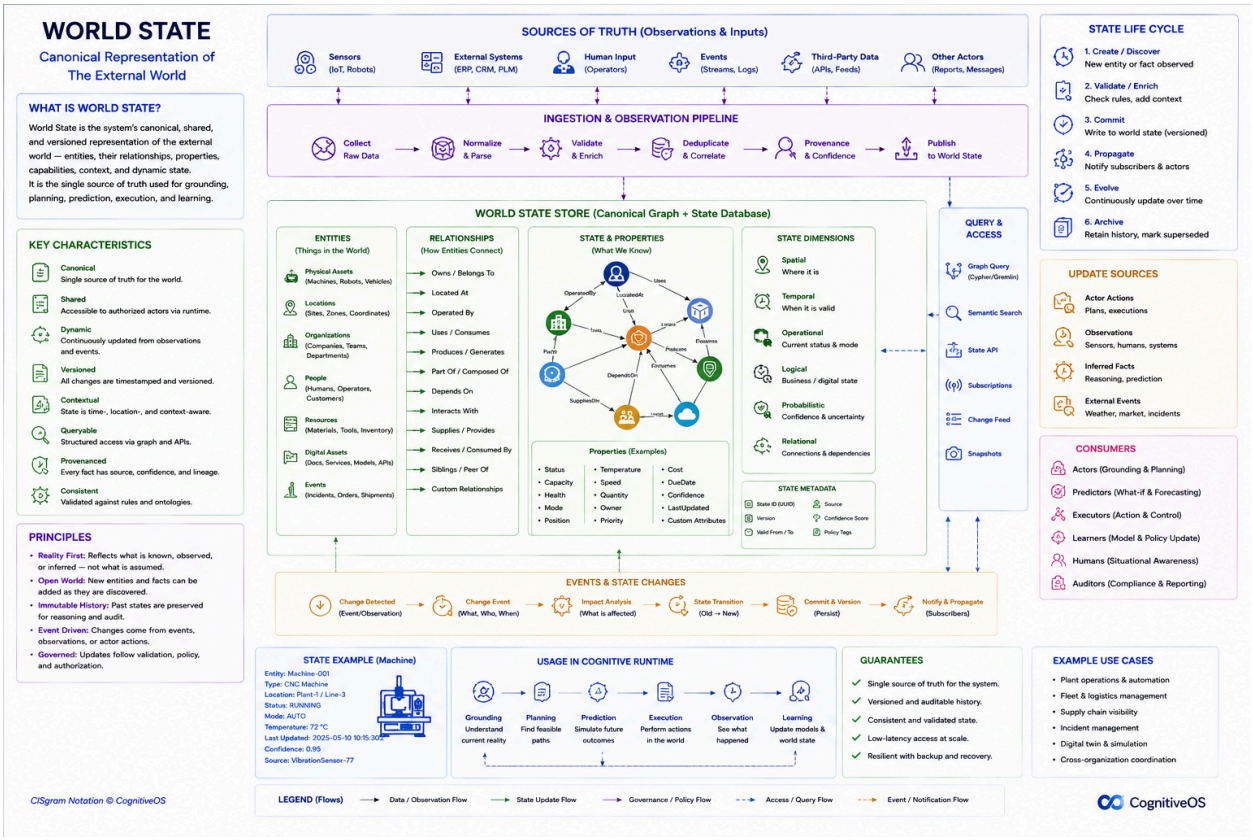
The Planetary Runtime provides:

- **Participant discovery** — discovering actors that are structurally eligible to participate in a coordination session.
- **Message routing** — reliably routing messages between actors.
- **Coordination sessions** — creating, maintaining, and terminating multi-actor coordination sessions.
- **Transaction state** — maintaining the execution and coordination state required for distributed interactions.
- **Observation and event propagation** — distributing relevant observations, events, and state changes to participating actors.
- **Coordination traces** — streaming the events and messages that constitute a coordination session for observability and auditability.
- **Actor scheduling** — scheduling when participating actors are given execution opportunities.
- **Reliable delivery** — providing delivery guarantees and handling retries, failures, and delivery state.

Operational Invariant

- **The Planetary Runtime** provides the execution arena.
- **The Society** defines the rules, affiliations, and trust topologies within that arena.
- **The Actor** exercises localized cognition and negotiation within those rules and infrastructure bounds

The Global World State



In cognitive science and autonomous systems engineering, a **world model** is an internal representation of an environment that enables an agent to simulate actions, predict future states, and reason under uncertainty. In a distributed multi-agent system, however, a single world model is insufficient. The Cognitive OS therefore separates environmental knowledge into two fundamentally different categories:

1. **Exogenous World Model — What is true:** The authoritative representation of the external world, including entities, relationships, locations, resources, events, and current state.
2. **Endogenous World Model — What is known or remembered:** The epistemic representation maintained by actors, including beliefs, semantic knowledge, historical experience, procedural knowledge, and episodic memory.

The **Global World State** is the canonical **exogenous world model** of the Cognitive OS. It provides the shared factual baseline against which all actors and societies ground their reasoning. The Global World State is the Cognitive OS's authoritative representation of external reality. It represents the system's best reconciled understanding of the world at a given point in time and serves as the common reference model for all actors. It is a **planetary resource** that continuously evolves as observations and World Perturbations are received, validated, reconciled, and applied by the Planetary Runtime.

The Global World State provides a consistent shared representation of:

- people;
- enterprises;
- societies;
- affiliations;
- memberships;
- relationships;
- physical entities;
- resources;
- environments;
- locations;
- events;
- policies; and
- other externally grounded world facts.

The Global World State answers questions such as:

- Does this resource exist?
- Where is this actor currently located?
- Is this machine available?
- Which actors belong to this enterprise?
- Which policies are currently active?
- What events have occurred?

The Global World State represents the external **state**, not the private cognitive state of an actor. An actor's beliefs, intentions, preferences, assumptions, memories, and hypotheses remain within its Local Belief Graph and cognitive systems.

Characteristics of the Exogenous World Model

Objective Real-Time Grounding

The Global World State provides the factual baseline required to answer temporal questions about the external world:

> **What is the state of reality at this point in time?**

It represents the latest reconciled state available to the Cognitive OS while preserving the temporal and provenance information required to understand how that state was established. The Global World State is not a static database. It continuously changes as the external world changes. World Perturbations such as machine failures, actor movement, inventory changes, relationship changes, resource availability, and policy updates act as state transitions over the world model.

Conceptually:

World State(t) + World Perturbation → World State(t+1)

The Planetary Runtime performs this transition rather than allowing individual actors to mutate the graph directly.

A **World Perturbation** represents an observation or event indicating that the external world has changed or may have changed.

Examples include:

- an actor enters a space;
- an actor exits a space;
- a machine changes state;
- inventory changes;
- a resource becomes available or unavailable;
- an actor changes location;
- a relationship changes;
- an affiliation changes;
- a membership changes;
- a policy becomes active or inactive;
- an external system reports a state change; or
- an observation provides new evidence about the state of the world.

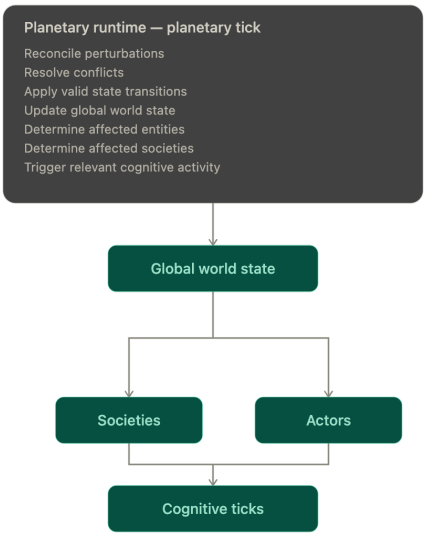
A World Perturbation is not necessarily equivalent to truth. It is an input to the world's reconciliation process. For example, two actors may report different states for the same machine. The Planetary Runtime must determine how those observations relate to one another using factors such as temporal ordering, provenance, authority, consistency, and applicable world-state rules. The resulting reconciled state is then incorporated into the Global World State.

The Planetary Runtime therefore does not assume that every observation can be immediately and blindly written into the Global World State. Instead, it continuously reconciles incoming World Perturbations. The objective is not instantaneous agreement between every observer.

The objective is to maintain the most accurate, provenance-aware, temporally consistent representation of the external world possible given the observations available to the system.

Planetary Tick

As previously mentioned there are two ways in which the world can change first due to an action taken by an actor that affects a few connected actors or due to a change at the world level affecting all actors or large societies of actors that large world level events are managed by the planetary tick . during a Planetary Tick, the Planetary Runtime performs the following sequence:



1. **Consume World Perturbations**
   Retrieve pending perturbations from the Perturbation Queue.
2. **Reconcile Perturbations**
   Validate, correlate, order, and reconcile incoming perturbations against the existing world state.
3. **Update the Global World State**
   Apply the resulting state transitions to the Global World Knowledge Graph.
4. **Identify Affected Entities**
   Determine which actors, resources, environments, relationships, or other entities are affected by the world change.
5. **Identify Affected Societies**
   Determine which societies are connected to the affected actors or entities through memberships, affiliations, relationships, or other relevant structures.
6. **Propagate the World Change**
   Make the resulting state change available to the relevant actors and societies.
7. **Trigger Cognitive Activity**
   Trigger cognitive ticks for actors or societies whose objectives, beliefs, plans, or responsibilities may have been affected by the change.

The Planetary Runtime therefore acts as the bridge between **changes in the external world** and **cognitive activity within the Cognitive OS**.The Planetary Runtime therefore does not decide what the affected actors should do. It ensures that the **change in reality reaches the cognitive entities whose decisions may depend upon it**.

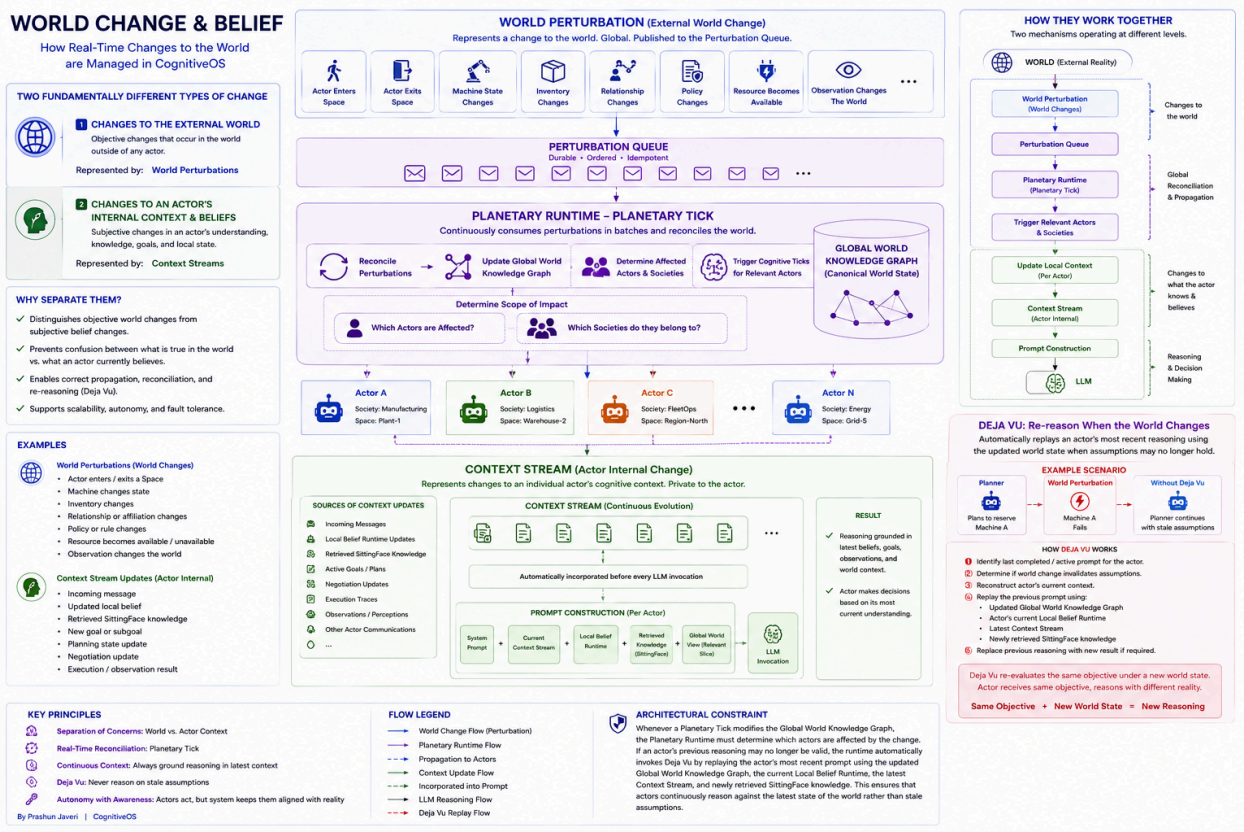
An actor may then retrieve the updated Global World State, update its Local Belief Graph, reassess its objective, and execute its own cognitive loop.

This establishes the fundamental flow:

> **World changes → Planetary Runtime reconciles → Global World State changes → affected actors and societies are identified → cognitive activity is triggered → actors reason and act.**

The Planetary Runtime consequently forms the boundary between **planetary reality and distributed cognition**. It maintains the shared exogenous model of the world while leaving interpretation, belief formation, planning, negotiation, and decision-making to autonomous actors.

World Change and Belief



CognitiveOS distinguishes between two orthogonal classes of state transitions:

- **World Change:** An externally observable modification to reality (e.g., a machine failing, inventory depleting, a resource relocating). World changes enter the system as **World Perturbations**, are reconciled by the Planetary Runtime, and update the **Global World State**.
- **Belief Change:** An internal update to an actor's mental model or **Local Belief Graph** (e.g., through direct observation, peer communication, or inference).

An actor's belief does not automatically constitute world truth, nor does a world change directly overwrite an actor's internal beliefs.

This separation allows multiple autonomous actors to maintain different beliefs about the same world while sharing a common external reference state.

A Belief Change occurs when an actor changes its internal representation of the world.

An actor may change its beliefs because:

- it observes the world directly;
- another actor provides information;
- the Global World State changes;
- an earlier belief is contradicted;
- new evidence becomes available;
- an inference changes;
- an event invalidates an assumption; or
- the actor learns from the outcome of an action.

Belief changes occur within the actor's Local Belief Graph. The Planetary Runtime does not directly modify an actor's beliefs. Instead, it propagates relevant changes in the Global World State, allowing each affected actor to independently determine how its beliefs should change.This is a feedback loop — objective state flows down to actors, and their actions flow back to mutate the world

The separation of reality and belief provides an objective reference framework for evaluating actor reliability. When an actor makes claims or observations that contradict the reconciled Global World State, the discrepancy serves as quantifiable evidence within the **Trust Network**:

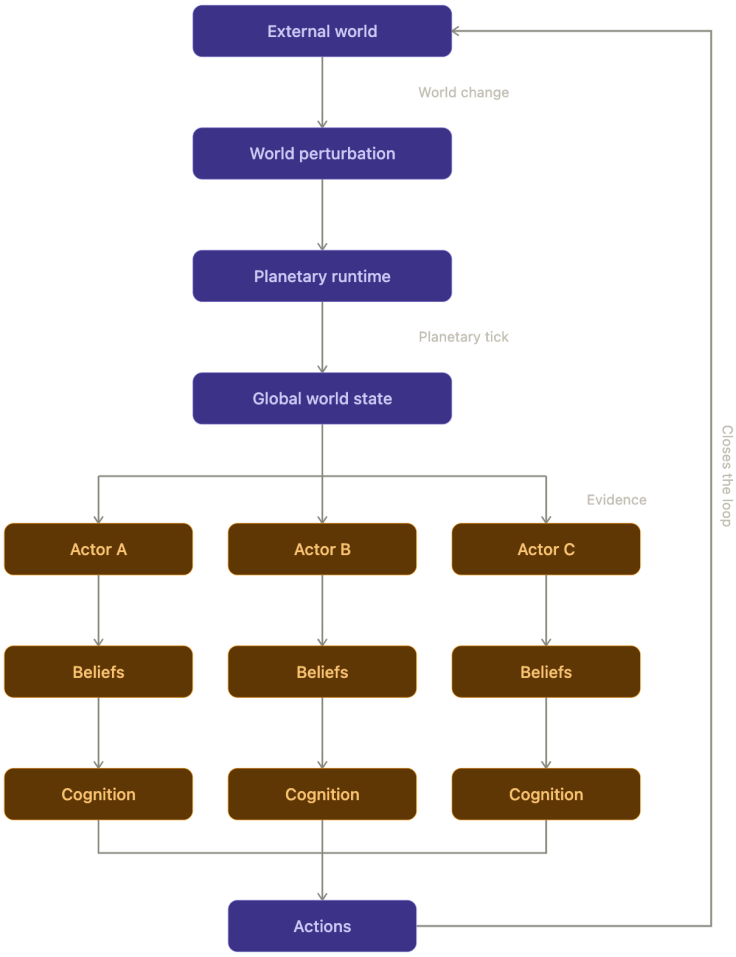
This mechanism ensures that while actors retain complete cognitive freedom to form subjective beliefs, their long-term reliability and societal standing remain anchored to objective reality.

A world change does not necessarily cause every actor to update its beliefs.Only actors for whom the change is relevant need to process it. For example, if a machine on Factory Floor A fails, an actor responsible for Factory Floor B may have no reason to update its local beliefs.

The Planetary Runtime therefore determines the potentially affected actors and societies, while the actors determine whether the change is cognitively relevant. This creates an important optimization and autonomy boundary:

> **The Planetary Runtime determines who may be affected; the actor determines what the change means.**

By maintaining a clean structural separation between **Reality** (Global World State), **Belief** (Local Belief Graphs), and **Experience** (accumulated interaction history), CognitiveOS enables multi-agent ecosystems to scale. Actors can independently reason, negotiate, harbor incomplete information, and recover from context loss while remaining safely grounded within a shared, authoritative representation of the external world.



While the **Global World State** tracks objective exogenous reality and **Local Belief Graphs** maintain persistent internal models, an actor actively executing a task requires a high-frequency, working-memory construct to process immediate stimuli.

When an actor engages in planning, negotiation, or multi-step execution, its operational context shifts continuously. Messages arrive, beliefs update, external constraints evolve, and knowledge

chunks are retrieved from *SittingFace*. Hardcoding these ephemeral elements into static prompts or forcing actors to poll global stores introduces latency and architectural coupling.

CognitiveOS resolves this via **Real-Time Context Streams**—private, dynamic working-memory streams bound exclusively to individual actors.

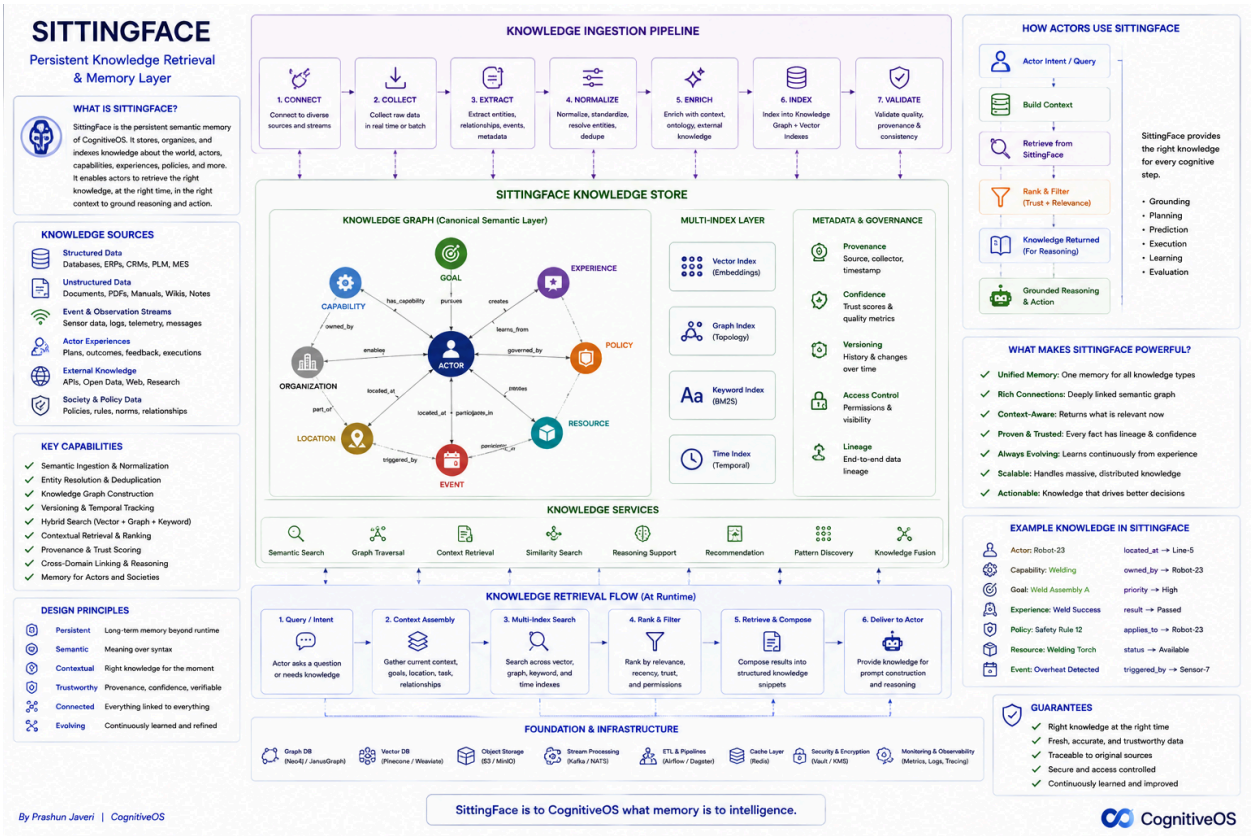
A Context Stream is strictly private to an individual actor. Throughout an execution lifecycle, it aggregates transient inputs and operational states, including:

- Incoming inter-actor messages and negotiation proposals.
- Recently updated local beliefs resulting from world change reconciliations.
- Retrieved semantic and episodic knowledge segments from *SittingFace*.
- Active goals, sub-goals, and current planning states.
- Execution traces, tool outputs, and environmental observations relevant to the actor.

> **The Global World State represents shared external reality. Local Belief Graphs represent actor-specific knowledge and interpretation. World Perturbations connect changes in reality to distributed cognition without directly controlling actor beliefs.**

This separation allows the Cognitive OS to support autonomous actors that can disagree, learn, negotiate, correct themselves, recover from context loss, and act independently while remaining grounded in a common representation of the world.

Sittingface and Prompt Compilation



SittingFace is the Cognitive OS's Retrieval-Augmented Generation (RAG) system.
Its purpose is to retrieve the most relevant knowledge required for reasoning before an LLM invocation. Unlike the Global World State, SittingFace is not the authoritative representation of reality.Instead, it is a knowledge retrieval system that indexes historical, semantic, episodic, and organizational knowledge to provide contextual grounding for cognitive reasoning.

SittingFace exists to answer questions such as:

- What have I seen before?
- Have I solved a similar problem?
- What conversations are relevant?
- Which experiences should influence my reasoning?
- What organizational knowledge applies here?
- Which constraints or policies are relevant?
- What information should be added to my prompt?

It is responsible for supplying context, not maintaining world state. SittingFace indexes knowledge from multiple sources, including:

- conversations
- observations
- experiences
- execution traces
- historical negotiations
- local beliefs
- planetary beliefs
- organizational knowledge
- enterprise knowledge
- society knowledge
- policies
- constraints
- relationships
- actor history
- learned preferences
- Documents

In the CognitiveOS architecture, separating exogenous environmental truth from endogenous knowledge retrieval is critical for maintaining systemic integrity. The Global World State and SittingFace represent two distinct, complementary memory and information stores. While the Global World State tracks objective, real-time external reality (what is true right now), SittingFace provides the historical, semantic, and episodic intelligence required to understand and solve problems within that reality (what is known or recorded).

- **Global World State (Exogenous Reality):** Answers the temporal query: *"What is currently true about the external world right now?"* It tracks objective facts, physical and logical resource states, and topological configurations updated synchronously by the Planetary Runtime.
- **SittingFace (Endogenous Epistemic Store):** Answers the query: *"What extra historical, procedural, or conceptual information do I need to solve the problem at hand?"* It acts as a Retrieval-Augmented Knowledge System (RAG) providing semantic depth, historical patterns, and domain expertise.

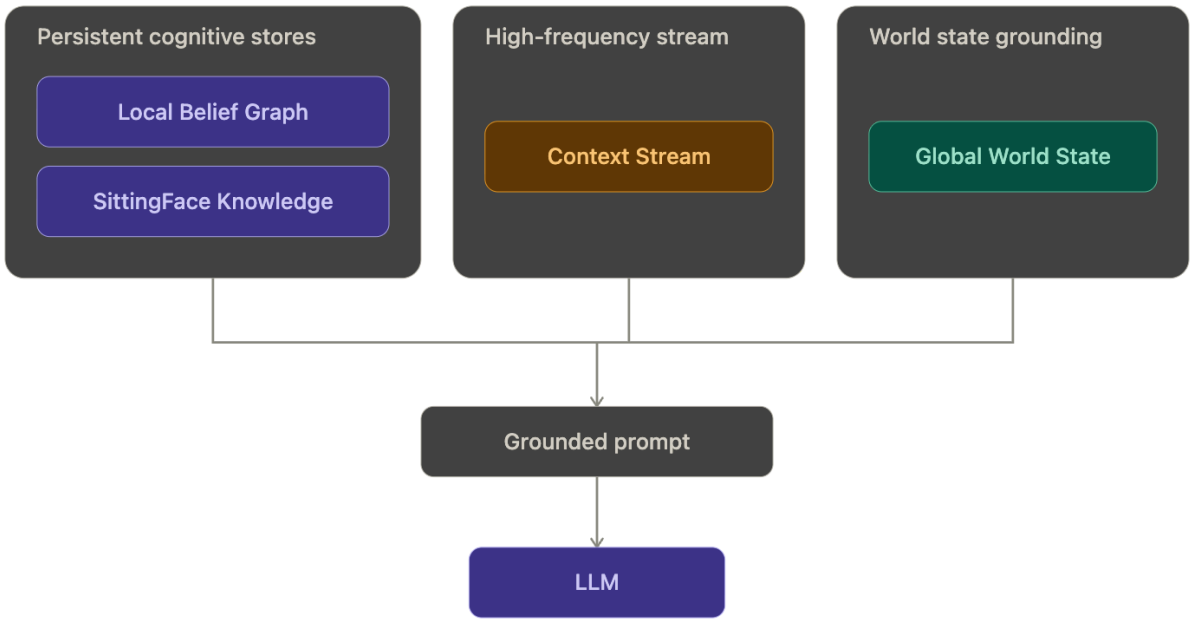
Traditional Large Language Model (LLM) agents maintain state primarily through expanding conversation history windows. As execution runs extend across complex, multi-actor workflows, this paradigm degrades: the model loses structural intent, conflates transient stimuli with durable knowledge, and fails to reconcile internal assumptions with external environmental mutations.

CognitiveOS decouples actor continuity from raw context windows by introducing the **Prompt Compilation Engine**. This subsystem draws from four distinct information sources organized across three architectural tiers, ensuring that an actor's prompt is structurally synthesized rather than arbitrarily concatenated.

Prompt Compilation

Before every LLM invocation, the CognitiveOS runtime compiles the actor's immediate working context by merging persistent beliefs with active stream updates and retrieved knowledge:

1. **Local Beliefs & SittingFace:** Long-term and persistent knowledge repositories supply the structural baseline.
2. **Context Stream Updates:** The private stream injects immediate, high-frequency operational changes (messages, execution traces, active negotiation states).
3. **Prompt Construction:** The runtime synthesizes these elements into a coherent working prompt.
4. **LLM Invocation:** The model reasons against this fully grounded, up-to-date context.



The compilation engine integrates data sources characterized by fundamentally different epistemic properties:

Tier 1: Persistent Cognitive Stores

Retained across individual inference cycles to form the actor's long-term cognitive baseline:

- **Local Belief Graph:** The actor's internal model of reality, including beliefs, assumptions, goals, constraints, hypotheses, and known relationships. It answers: *What does this actor currently believe about the world and its objective?*

- **SittingFace Knowledge Store:** A Retrieval-Augmented Knowledge System housing semantic memory, episodic recall, procedural documentation, historical precedents, and domain expertise. It answers: *What does this actor know or remember that is relevant to the current objective?*

Tier 2: High-Frequency Operational Stream

- **Context Stream:** Transient, short-lived runtime events captured immediately prior to the inference cycle. It includes incoming messages, negotiation updates, recent tool results, active transaction states, and immediate observations. It answers: *What is happening right now?*

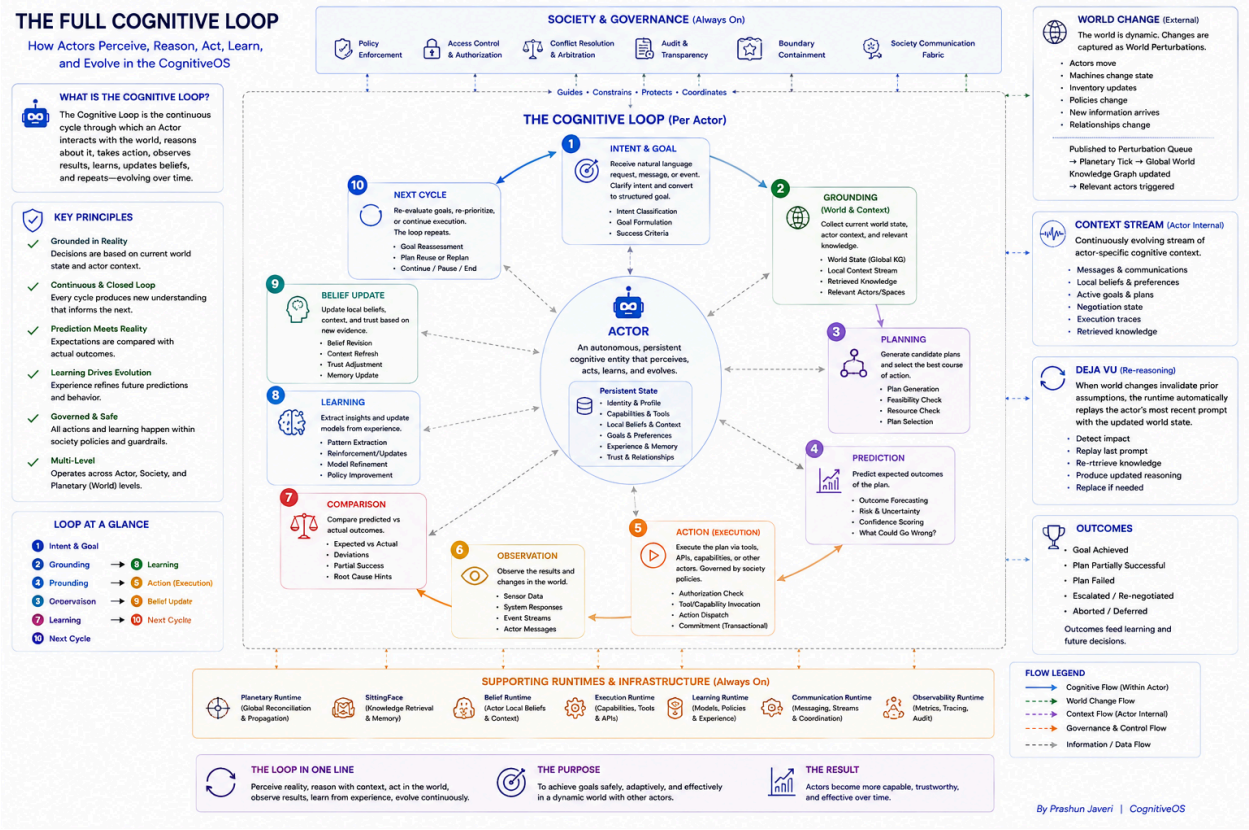
Tier 3: Global World State Grounding

- **Global World State :** The authoritative, reconciled external reality of the CognitiveOS ecosystem. Retrieved via graph traversal based on the actor's objective, it includes entity states, resource availability, policies, and spatial parameters. It answers: *What is currently true about the external world that constrains this decision?*

Because prompts are assembled deterministically from persistent storage layers rather than linear transcript logs, actors possess native resilience to context loss. Interrupted or restarted actors can reconstruct their full operational reality

> **The Prompt Compilation Invariant:** Belief, memory, operational context, and external reality must remain distinguishable throughout prompt construction. The compiler must never erase their provenance, ensuring the language model maintains explicit clarity regarding what it believes, what it remembers, what is happening now, and what external reality dictates as true.

3. The Closed Cognitive Runtime



Moving from an abstract architectural concept to an operational system requires more than assembling loosely coupled microservices for planning, memory, and execution. The defining challenge is maintaining a single, persistent, governed cognitive process whose state evolves seamlessly as the system interacts with the world.

The runtime begins with an interpreted intention and its associated goal, grounds that goal against available knowledge and current system state, and produces a plan. The plan is then subjected to additional runtime controls before being compiled into an executable representation. Prediction evaluates possible outcomes before execution, while execution produces observable evidence about what occurred in the external system. What this does is provide the predicted vs actual state at time (n+1)

The resulting observation is not treated as an implicit indication of success. It is passed to an explicit comparison stage in which expected and actual outcomes can be evaluated. This distinction allows the runtime to represent successful execution, failure, partial outcomes, and insufficient evidence as distinct states rather than collapsing all execution into a binary success signal. The Comparator consequently becomes an evidence boundary between what the system intended to happen and what the system can establish actually happened. The qualification baseline reports verification of expected and actual state, deterministic normalization, node-level differences, partial outcomes, provenance, and epistemic loss within this comparison process.

The comparison result then becomes an input to learning. Learning is not inferred merely from the existence of an execution record. Instead, the runtime uses comparator evidence to distinguish successful transitions, failed transitions, and partial execution. Unexecuted portions of a plan are not incorrectly interpreted as failures. The learned state is persisted and subsequently made available to future prediction. This creates a causal progression from execution evidence to future cognitive behavior:

Execution → Comparator → Learning → Persisted Learned State → Future Prediction.

The runtime is therefore not simply accumulating execution history. It is maintaining an evolving relationship between **what the system intended, what it knew, what it predicted, what it did, what actually happened, what it learned, and what it subsequently believed**.

This distinction becomes particularly important under failure. A conventional agent loop may interpret a failed tool invocation as an instruction to retry or ask a language model what to do next. CognitiveOS instead preserves the execution state and evaluates the outcome as evidence. In the qualified runtime, execution includes explicit lifecycle transitions, dependency gating, checkpointing, restoration, and rejection of illegal state transitions.

Persistence and recovery provide another critical dimension of the runtime. This allows the cognitive process to survive interruption rather than requiring the system to reconstruct its state solely from prompts or conversation history.Taken together, these capabilities establish a substantially different architectural property from that of a conventional agent. The system is not merely capable of producing a sequence of actions but able to generate and execute plans that are grounded in real state with the ability to learn and self correct . the complete workflow is shown below

Separation of Responsibilities and final workflow

Originating Actor	Planetary Runtime
Defines the objective	Coordinates communication
Selects participants	Discovers eligible participants
Constructs grounded prompts	Routes messages
Executes the cognitive loop	Schedules execution

Negotiates and reasons	Tracks coordination sessions
Aggregates responses	Streams traces and events
Decides next actions	Manages propagation and transaction metadata

